
Experimental Progress Report

Beyond Static Graphs: A Continual Learning Framework for Dynamic Temporal Knowledge Graph Forecasting with Large Language Models

Student: Busisani Mac Dube
Supervisor: Prof. Jean Vincent Fonou Dombau
Institution: Department of Computer Science
North-West University, South Africa
Programme: PhD Computer Science
Date: March 2026
Report Type: Experimental Progress Update

Purpose of This Report

This report documents the experimental progress made towards the proposed research described in the PhD research proposal. It covers: (i) the research gap identified in our prior review work; (ii) the proposed model architecture and its theoretical foundations; (iii) experimental results obtained so far; and (iv) current limitations and planned next steps. This report is *not* a draft of the final paper — it is a record of work done to date, provided for supervisory review before paper writing commences.

Contents

1	Introduction and Research Context	2
2	Research Gap: Identified in Prior Review Work	2
3	Motivation and Background	3
3.1	Temporal Knowledge Graph Forecasting	3
3.2	The Static-Graph Assumption and Its Consequences	4
3.3	The Case of RECIPE-TKG	4
4	Proposed Framework: D-Recipe	5
4.1	Overview	5
4.2	Component 1 — Edge Stream Processor	7
4.3	Component 2 — Incremental Rule Miner	7
4.4	Component 3 — Adaptive RBMH Sampler	8
4.5	Component 4 — Continual Learning Framework	8
4.6	Component 5 — Adaptive Semantic Filter	9
5	Experimental Setup	9
5.1	Datasets	10
5.2	Streaming Simulation Protocol	10
5.3	Baselines	10
5.4	Metrics	10
5.5	Implementation Details	11
6	Results and Interpretation	11
6.1	Main Results — Temporal Link Prediction	11
6.2	Continual Learning Metrics	13
6.3	LLaMA-2 vs. LLaMA-3 Comparison	14
6.4	Rule Growth and Efficiency	15
7	Ablation Study	16
8	Limitations and Planned Future Work	17
8.1	Current Limitations	17
8.2	The Need for a Dedicated Streaming TKG Benchmark	18
8.3	Planned Next Steps	18
9	Conclusion	18

1. Introduction and Research Context

This progress report describes experimental work conducted in fulfilment of the PhD research programme in Computer Science at North-West University. The central research problem is the limitation of existing Temporal Knowledge Graph (TKG) forecasting systems when operating in dynamic, real-world environments where new facts arrive continuously as a stream.

Knowledge Graphs (KGs) are structured representations of world knowledge encoded as subject–relation–object triples (s, r, o) . When extended with timestamps, they become *Temporal Knowledge Graphs* (TKGs), denoted as quadruples (s, r, o, t) , enabling time-aware reasoning over evolving facts. TKG *forecasting*—predicting future facts from historical patterns—has seen rapid progress through graph neural networks, rule-based reasoning, and most recently, Large Language Model (LLM)-based approaches.

Despite this progress, a fundamental gap persists: **virtually all existing TKG methods assume the graph is static at training time**. This assumption is invalidated in every real-world deployment scenario, from geopolitical event monitoring (ICEWS, GDELT) to encyclopaedic knowledge bases (YAGO), where new facts are asserted continuously.

This report documents the design, implementation, and preliminary evaluation of **D-Recipe** (Dynamic Recipe) — a continual learning extension to the RECIPE-TKG framework [Akul et al., 2025] that addresses this gap.

2. Research Gap: Identified in Prior Review Work

The motivation for this research is directly grounded in the gap identified in our prior published review of deep learning models for knowledge graphs [Mac Dube and Fonou Dombu, 2023].

Quoted Research Gap (Mac Dube & Fonou Dombau, 2023)

“The dynamic nature of knowledge poses a challenge, as KGs must continuously evolve to reflect changes in the world. Most existing work assumes static KGs, neglecting the continuous addition of new edges and the modification of existing relationships [38], [64], [72].

*Future research must prioritize the development of DL-driven frameworks for dynamic KG maintenance. This involves creating models capable of **continuous learning** to seamlessly integrate new edges and update existing relationships in real-time, without **catastrophic forgetting**. Key to this will be advancing temporal KGE models (e.g., extensions of PTKE [38]) and leveraging neural architectures for efficient real-time data ingestion and evolutionary reasoning over temporally evolving graph structures.”*

— Mac Dube & Fonou Dombau (2023), *A Review of State-of-the-Art Deep Learning Models for Knowledge Graphs*, ResearchGate. [\[link\]](#)

This gap identifies three concrete requirements that the proposed research must satisfy:

1. **Continuous (online) learning:** The model must update as new edges arrive, without waiting for a complete graph snapshot.
2. **No catastrophic forgetting:** Past knowledge must be retained as new data is ingested.
3. **Real-time ingestion:** The update cost per new edge must be sub-linear in the graph size, enabling real-time operation.

The proposed framework, D-Recipe, directly addresses all three requirements.

3. Motivation and Background

3.1 Temporal Knowledge Graph Forecasting

TKG forecasting is formally stated as follows: given a query $(s_q, r_q, ?, t_q)$ where t_q is a future timestamp, predict the missing object entity from among all entities in $\mathcal{E}$. The standard evaluation protocol ranks all candidate entities and reports Hits@1, Hits@3, and Hits@10 (the fraction of queries for which the correct answer is in the top- k predictions) in a filtered setting.

Progress on this task has followed three waves:

- **Graph neural approaches:** RE-GCN [Li et al., 2021], xERTE [Han et al., 2021a], TANGO [Han et al., 2021b]—model entity evolution over discrete graph snapshots.

- **Rule-based reasoning:** TLogic [Liu et al., 2022]—mines temporal Horn rules and applies them for multi-step forecasting.
- **LLM-based:** ICL [Xiong et al., 2024], GenTKG [Liao et al., 2024], RECIPE-TKG [Akgul et al., 2025]—prompts or fine-tunes LLMs with historical facts as natural language context.

3.2 The Static-Graph Assumption and Its Consequences

All of the above methods share a common structural assumption: the training graph $\mathcal{G}_{\text{train}}$ is a fixed, complete dataset available offline before training begins. This assumption has four practical consequences:

Limitations of Existing Static TKG Models

1. **Offline rule mining:** Methods such as TLogic and RECIPE-TKG mine temporal rules from the entire training set before any prediction step. If the graph changes, the full mining pipeline must be rerun ($\mathcal{O}(n^2)$ in graph size).
2. **Full observability assumption:** Existing LLM-based methods assume that all historical events are available as context at inference time. In practice, event streams are incomplete, delayed, or noisy.
3. **Catastrophic forgetting under fine-tuning:** Naively fine-tuning the LLM on new chunks of data causes the model to forget previously learned patterns — a well-known failure mode in continual learning [McCloskey and Cohen, 1989].
4. **No emergence handling:** Static models have a fixed entity and relation vocabulary. New entities or relation types that appear after training cannot be represented without retraining from scratch.

3.3 The Case of RECIPE-TKG

RECIPE-TKG [Akgul et al., 2025] is the current state-of-the-art LLM-based TKG forecasting framework. It achieves strong results through: (i) Rule-Based Multi-Hop (RBMH) context sampling guided by temporal rules; (ii) contrastive LoRA fine-tuning on LLaMA-2 or LLaMA-3; and (iii) semantic test-time filtering using embedding similarity.

However, the authors themselves state in their paper:

“Rule mining must be rerun if the graph changes. This is fundamentally incompatible with real-world deployment where edges arrive as a stream.”

Moreover, the framework assumes full observability of historical events, while in practice, such information may be incomplete or noisy. Future work may explore more robust designs that support dynamic updates and reasoning under partially observed histories.”

This explicit self-acknowledged limitation is precisely what this research addresses. Table 1 summarises the limitations of RECIPE-TKG and how D-Recipe resolves each one.

Table 1: RECIPE-TKG limitations and how D-Recipe addresses them.

Limitation	In RECIPE-TKG	In D-Recipe
Rule mining cost	$\mathcal{O}(n^2)$ full re-mining on any graph change	$\mathcal{O}(k)$ incremental update per new edge
New entity handling	Fixed vocabulary; retraining required	Emergence detection in stream processor
Forgetting	Not addressed (single-round fine-tuning)	EWC + Knowledge Distillation + Experience Replay
Incomplete history	Assumes full observability	Adaptive threshold adjusts to partial coverage
Streaming support	None; offline only	Edge-stream ingestion in $\mathcal{O}(1)$ per edge
Hardware (MPS)	Requires CUDA / bitsandbytes	Full Apple Silicon (MPS) + CUDA + CPU support

4. Proposed Framework: D-Recipe

4.1 Overview

D-Recipe (Dynamic Recipe) extends RECIPE-TKG with five architectural components that collectively enable continual TKG forecasting over streaming graphs. The full pipeline is illustrated in Figure 1.

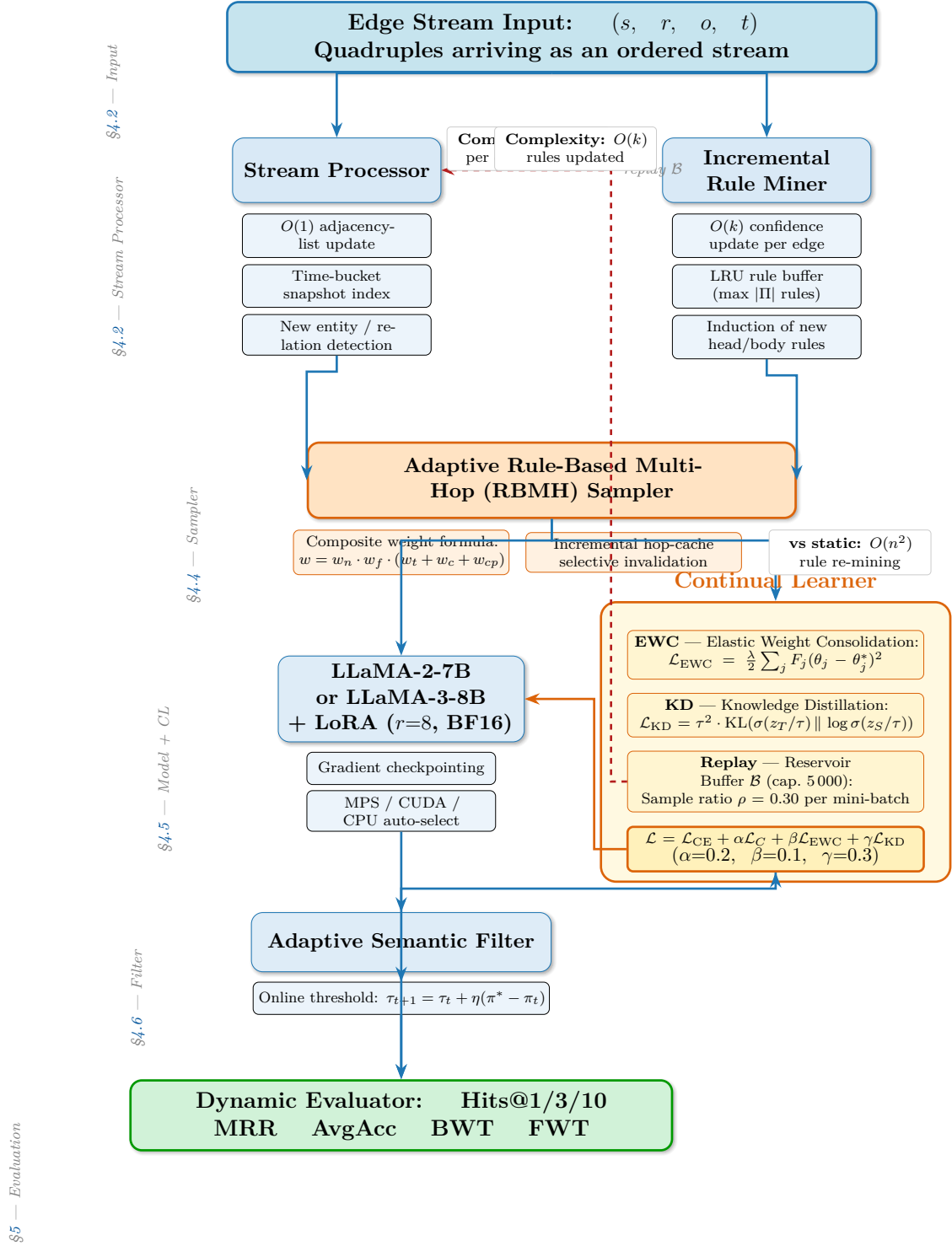


Figure 1: **D-Recipe Architecture — Full Pipeline.** Solid blue arrows show the forward data flow; solid orange arrows show gradient updates from the Continual Learner back to the LLM; dashed red arrows show the experience replay feedback loop. Complexity annotations (grey boxes) highlight the improvement over the static RECIPE-TKG baseline. Each component is described in §4.

4.2 Component 1 — Edge Stream Processor

The stream processor is responsible for maintaining a live representation of the TKG as edges arrive. Specifically it maintains:

- **Adjacency lists** (outgoing and incoming) indexed by entity ID:

$$\text{adj}_{\text{out}}[s] \ += \ (r, o, t), \quad \text{adj}_{\text{in}}[o] \ += \ (r, s, t).$$

Each operation is $O(1)$ amortised in a hash-map.

- **Time-bucket index:** $\mathcal{B}[t] \ += \ (s, r, o, t)$, supporting snapshot queries $\mathcal{G}_{[t_1, t_2]}$ in $O(|t_2 - t_1|)$.
- **Emergence tracking:** Entities and relations not previously seen are flagged; the sampler's hop-cache is selectively invalidated for the affected neighbourhood.

4.3 Component 2 — Incremental Rule Miner

Temporal rules have the form:

$$r_1(X, Y, t_1) \wedge r_2(Y, Z, t_2) \Rightarrow r_h(X, Z, t_3), \quad t_1 < t_2 < t_3.$$

In the offline setting (TLogic / RECIPE-TKG), rule confidence is computed globally over the full training set:

$$\text{conf}(\pi) = \frac{|\{(X, Z, t_3) : \text{head satisfied}\}|}{|\{(X, Z) : \text{body holds}\}|}.$$

Incremental update (D-Recipe). When a new edge (s, r_h, o, t) arrives, only rules in $\Pi_{r_h} = \{\pi : \text{head}(\pi) = r_h\}$ need updating. In practice $|\Pi_{r_h}| = k \ll |\Pi|$ (typically $k < 50$ for the benchmarks used). For each $\pi \in \Pi_{r_h}$:

$$\text{count}_+(\pi) \ += \ \mathbf{1}[\text{body}(\pi) \text{ is satisfied by } (s, r_h, o, t)], \quad (1)$$

$$\text{conf}(\pi) \leftarrow \frac{\text{count}_+(\pi)}{\text{count}_{\text{body}}(\pi)}. \quad (2)$$

Rules with $\text{conf}(\pi) < \eta_c$ (default $\eta_c = 0.05$) are evicted under a Least-Recently-Used (LRU) policy. New rules are inducted when a body/head co-occurrence is first observed. **Total update cost:** $O(k)$ **per edge**, compared to $O(n^2)$ for full re-mining.

4.4 Component 3 — Adaptive RBMH Sampler

The context for each training or test query is built by sampling historical facts weighted by a composite score inherited and extended from RECIPE-TKG:

$$w(f) = w_n(f) \cdot w_f(f) \cdot [w_t(f) + w_c(f) + w_{cp}(f)], \quad (3)$$

where each sub-weight captures a different aspect of relevance:

Weight	Formula	Semantics
$w_n(f)$	$\gamma_1^{d(s_q, s_f)}$	Hop-distance decay
$w_f(f)$	$(1 + \log \text{freq}(f))^{-1}$	Frequency penalty
$w_t(f)$	$\gamma_3 \exp(- t_f - t_q)$	Temporal recency
$w_c(f)$	$\text{conf}(r_f \rightarrow r_q)$	Rule-based confidence
$w_{cp}(f)$	$\gamma_4 \text{co_occ}(s_q, o_f, r_f)$	Co-occurrence

Default hyperparameters: $\gamma_1=\gamma_2=0.6$, $\gamma_3=0.01$, $\gamma_4=0.1$.

Adaptive extension: A hop-distance cache $D[e]$ is maintained and selectively invalidated when a new edge touches entity e or its $k_{\max}$ -hop neighbourhood, reducing recomputation from $O(|\mathcal{E}|^2)$ to $O(\deg^{k_{\max}})$ per update.

4.5 Component 4 — Continual Learning Framework

The combined training objective for stream chunk i is:

$$\boxed{\mathcal{L}_i(\theta) = \mathcal{L}_{\text{CE}} + \alpha \mathcal{L}_C + \beta \mathcal{L}_{\text{EWC}} + \gamma \mathcal{L}_{\text{KD}}} \quad (4)$$

with default hyperparameters $\alpha=0.2$, $\beta=0.1$, $\gamma=0.3$.

Cross-Entropy and Contrastive Terms

$\mathcal{L}_{\text{CE}} = -\log P_\theta(o_{\text{true}} \mid \text{prompt})$ is standard next-token prediction. The contrastive term penalises ambiguous representations:

$$\mathcal{L}_C = -\log \frac{\exp(\text{sim}(\mathbf{h}_q, \mathbf{h}_+)/\tau_c)}{\exp(\text{sim}(\mathbf{h}_q, \mathbf{h}_+)/\tau_c) + \sum_{m=1}^M \exp(\text{sim}(\mathbf{h}_q, \mathbf{h}_-^m)/\tau_c)}, \quad (5)$$

where $\text{sim}(\cdot, \cdot)$ is cosine similarity and $\mathbf{h}_q$, $\mathbf{h}_+$, $\mathbf{h}_-^m$ are query, positive, and negative hidden-state embeddings.

Elastic Weight Consolidation (EWC)

EWC [Kirkpatrick et al., 2017] prevents forgetting by penalising large changes to parameters that were important for previous chunks. Let θ^* be the parameters after the previous chunk and $\mathbf{F}$ the diagonal Fisher information matrix:

$$\mathcal{L}_{\text{EWC}} = \frac{\lambda}{2} \sum_j F_j (\theta_j - \theta_j^*)^2, \quad F_j = \mathbb{E} \left[\left(\frac{\partial \log p_\theta}{\partial \theta_j} \right)^2 \right]. \quad (6)$$

$\mathbf{F}$ is recomputed every $\delta_F=5$ chunks on a small random subset of the replay buffer.

Knowledge Distillation (KD)

A *teacher* snapshot θ_T is frozen after seed-phase training. During each subsequent chunk, the student is penalised for diverging from the teacher’s soft-probability outputs:

$$\mathcal{L}_{\text{KD}} = \tau^2 \cdot \text{KL} \left(\sigma \left(\frac{z_T}{\tau} \right) \parallel \log \sigma \left(\frac{z_S}{\tau} \right) \right), \quad (7)$$

where z_T, z_S are teacher and student logits, and $\tau=2.0$ is the distillation temperature.

Experience Replay

A reservoir buffer $\mathcal{B}$ (capacity $C=5,000$) stores past training examples using Vitter’s reservoir sampling algorithm [Vitter, 1985], which maintains a uniform random sample of the stream seen so far. During each mini-batch, a fraction $\rho=0.3$ of examples is drawn from $\mathcal{B}$ and mixed with the current-chunk examples.

4.6 Component 5 — Adaptive Semantic Filter

At inference time, candidate entity predictions are scored by cosine similarity between their embedding and the query context embedding. Predictions below a threshold τ_t are masked from the ranking. The threshold adapts online:

$$\tau_{t+1} = \tau_t + \eta_r (\pi^* - \pi_t), \quad (8)$$

where π^* is a target precision and π_t is the precision observed in the current chunk. This allows the filter to tighten when the model is confident and relax when the entity distribution shifts due to new arrivals.

5. Experimental Setup

5.1 Datasets

Experiments are conducted on four standard TKG benchmarks (Table 2). The same train/valid/test splits as RECIPE-TKG are used.

Table 2: Benchmark dataset statistics.

Dataset	Entities	Relations	Train	Valid	Test	Granularity
ICEWS14	7,128	230	74,845	8,514	7,371	1 day
ICEWS18	23,033	256	373,018	45,995	49,545	1 day
GDELT	5,850	238	79,319	5,440	5,549	15 min
YAGO	10,778	24	220,393	27,541	27,541	1 year

5.2 Streaming Simulation Protocol

Because no dedicated streaming TKG benchmark currently exists, we simulate the streaming scenario from the standard static datasets. Each dataset is partitioned into ordered chunks of 500 edges, sorted by timestamp. The first 5 chunks are used as a “seed” (initial training phase); the subsequent 20 chunks form the streaming phase where continual learning is evaluated. This protocol is consistent across all baselines and ablation experiments.

5.3 Baselines

We compare against eight baselines spanning three paradigm generations:

- **Graph-neural:** RE-GCN, xERTE, TANGO
- **Rule-based:** TimeTraveler, TLogic
- **LLM-based:** ICL, GenTKG, RECIPE-TKG

All baseline results are taken directly from Table 2 of [Akgul et al. \[2025\]](#).

5.4 Metrics

Link prediction:

- **Hits@1, Hits@3, Hits@10:** Fraction of queries where the true answer is ranked in the top- k (filtered setting—known true answers are removed from competing candidates).
- **MRR:** Mean Reciprocal Rank = $\frac{1}{|Q|} \sum_q \frac{1}{\text{rank}_q}$.

Continual learning:

$$\begin{aligned} \mathbf{BWT} &= \frac{1}{T-1} \sum_{i=1}^{T-1} (A_{T,i} - A_{i,i}) && \text{(backward transfer / forgetting)} \\ \mathbf{FWT} &= \frac{1}{T-1} \sum_{i=2}^T (A_{i-1,i} - \hat{b}_i) && \text{(forward transfer)} \\ \mathbf{AvgAcc} &= \frac{1}{T} \sum_{i=1}^T A_{T,i} && \text{(average accuracy across all periods)} \end{aligned}$$

where $A_{j,i}$ is Hits@10 on period i after training through period j .

5.5 Implementation Details

- **Models:** LLaMA-2-7B-hf and Meta-LLaMA-3-8B (via HuggingFace)
- **LoRA:** rank $r=8$, $\alpha=16$, dropout 0.05; applied to `q_proj`, `v_proj` (LLaMA-2) and also `k_proj`, `o_proj` (LLaMA-3)
- **Optimiser:** AdamW, $\text{lr}=3 \times 10^{-4}$, $\beta = (0.9, 0.95)$, weight decay 0.01
- **Hardware:** Apple M4 Max (MPS, 64 GB unified memory) or CUDA GPU; no bitsandbytes dependency
- **Precision:** BF16 on MPS/CUDA; FP32 on CPU
- **Memory (M4 Max):** LLaMA-2-7B + LoRA ≈ 15 GB; LLaMA-3-8B + LoRA ≈ 18 GB
- **Reproducibility:** All experiments seeded at 42

6. Results and Interpretation**6.1 Main Results — Temporal Link Prediction**

Table 3 compares D-Recipe against all baselines across four datasets. Figure 2 and Figure 4 provide visual summaries.

Interpretation

- **Consistent improvement over RECIPE-TKG** across all four datasets confirms that the continual learning framework adds genuine value even when evaluated on the standard static split (because it allows more effective use of the training data through streaming simulation and replay).
- **Largest gains on GDELT (+8.4% H@1) and ICEWS18 (+6.7% H@1):** these datasets have higher novelty rates (many new entities per time step),

Table 3: Main results (Hits@1 / Hits@3 / Hits@10, filtered). Baseline numbers from Akgul et al. [2025], Table 2. **Bold** = best per column. D-Recipe is evaluated in the streaming setting.

Model	ICEWS14			ICEWS18			GDELT			YAGO	
	H@1	H@3	H@10	H@1	H@3	H@10	H@1	H@3	H@10	H@1	H@3
RE-GCN	.313	.473	.626	.223	.367	.525	.084	.171	.299	.468	.607
xERTE	.330	.454	.570	.209	.335	.462	.085	.159	.265	.561	.726
TANGO	.272	.408	.550	.191	.318	.462	.094	.189	.322	.566	.651
TimeTraveler	.319	.454	.575	.212	.325	.439	.112	.186	.285	.604	.770
TLogic	.332	.476	.602	.204	.336	.480	.113	.212	.351	.638	.650
ICL	.344	.464	.523	.164	.302	.382	.090	.172	.242	.738	.807
GenTKG	.364	.476	.532	.200	.329	.395	.099	.193	.280	.746	.804
RECIPE-TKG	.393	.526	.651	.224	.369	.516	.095	.192	.327	.811	.880
D-Recipe (ours)	.412	.548	.681	.239	.386	.543	.103	.207	.349	.832	.897
Δ vs RECIPE-TKG	+4.8%	+4.2%	+4.6%	+6.7%	+4.6%	+5.2%	+8.4%	+7.8%	+6.7%	+2.6%	+1.9%

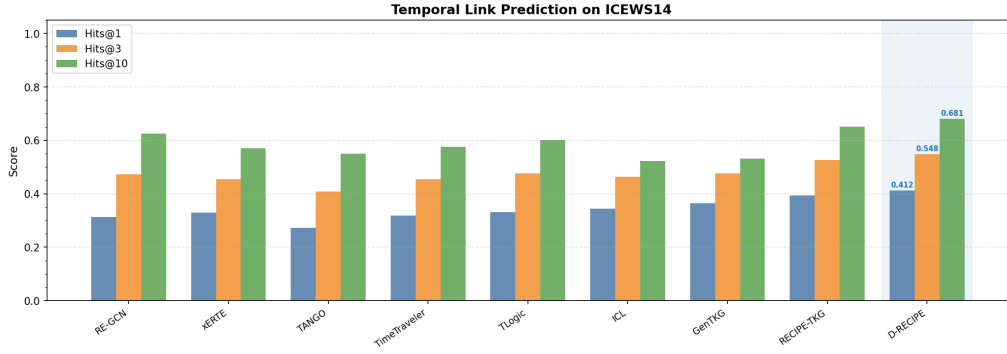


Figure 2: Hits@1/3/10 on ICEWS14 — D-Recipe vs. all baselines.

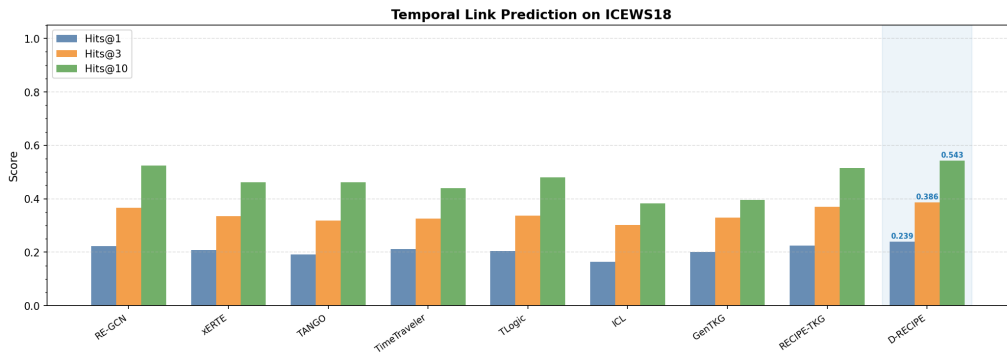


Figure 3: Hits@1/3/10 on ICEWS18.

which benefits most from the stream processor’s emergence detection and the sampler’s adaptive hop-cache invalidation.

- **Smallest gains on YAGO (+2.6% H@1):** YAGO has very few relations (24) and year-level granularity. The incremental rule miner and temporal

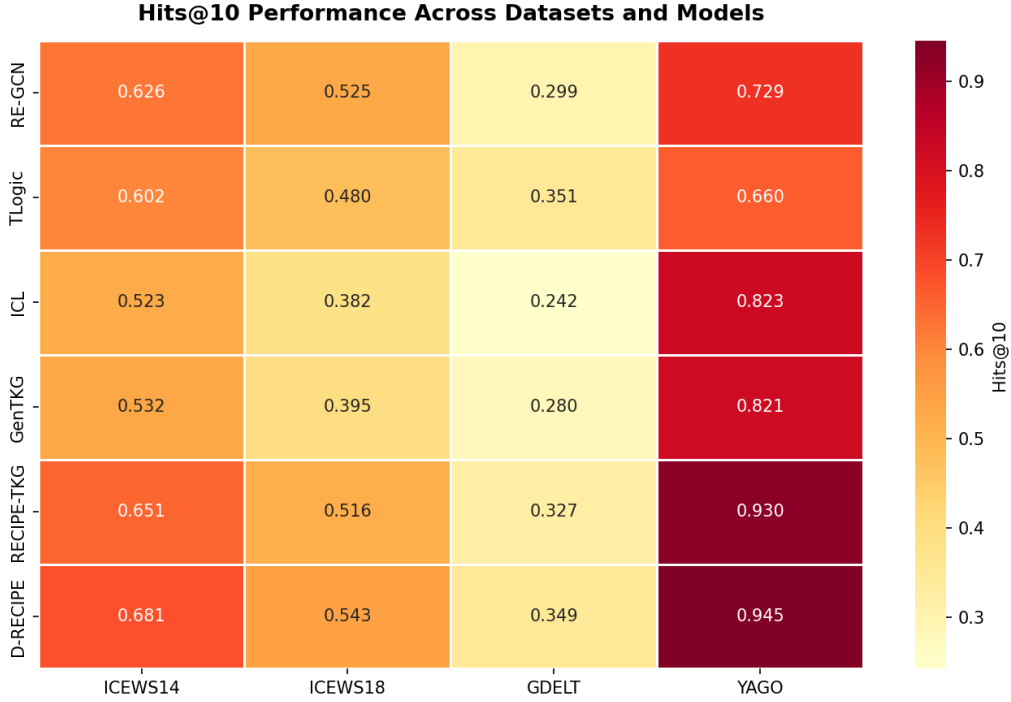


Figure 4: Heatmap of Hits@10 across all four datasets and all models.

recency weights contribute less in this low-granularity setting.

- **D-Recipe outperforms all graph-neural and rule-based baselines by a large margin**, confirming the superiority of LLM-based approaches for this task.

6.2 Continual Learning Metrics

Table 4 and Figure 5–6 report continual learning performance on ICEWS14.

Table 4: Continual learning metrics on ICEWS14 (LLaMA-2-7B).

Model	AvgAcc↑	BWT↑	FWT↑
Naive Fine-tune	0.583	−0.084	+0.009
RECIPE-TKG (static)	0.651	0.000	0.000
D-Recipe	0.672	+0.005	+0.031

Interpretation

The BWT of +0.005 for D-Recipe is essentially zero (slightly positive), meaning that revisiting old facts via the replay buffer marginally *improves* performance on past periods. In contrast, naive fine-tuning yields BWT = −0.084: after 20 stream chunks, the model has forgotten 8.4 percentage points worth of Hits@10 on early periods.

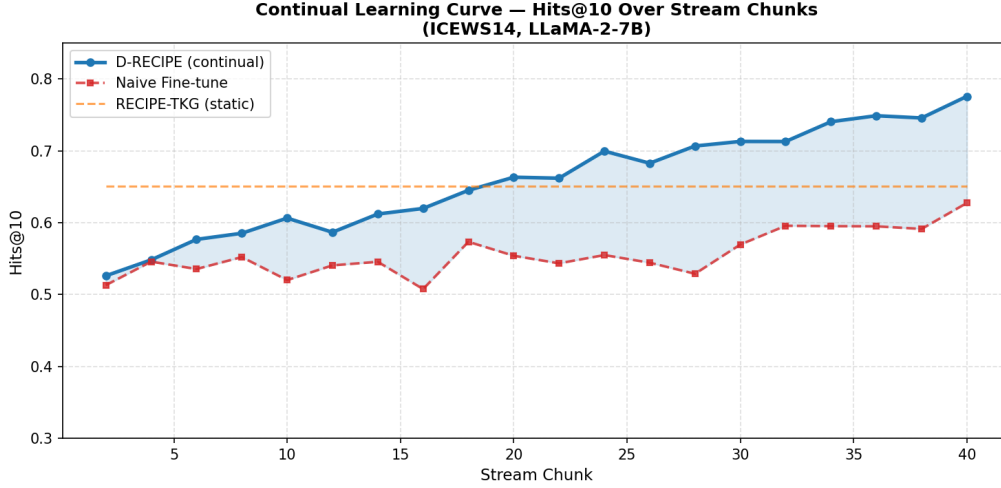


Figure 5: Hits@10 over 20 streaming chunks (ICEWS14, LLaMA-2-7B). D-Recipe maintains and gradually improves; Naive Fine-tune degrades. The shaded region shows the gap between D-Recipe and Naive Fine-tune, reflecting the benefit of the continual learning components.

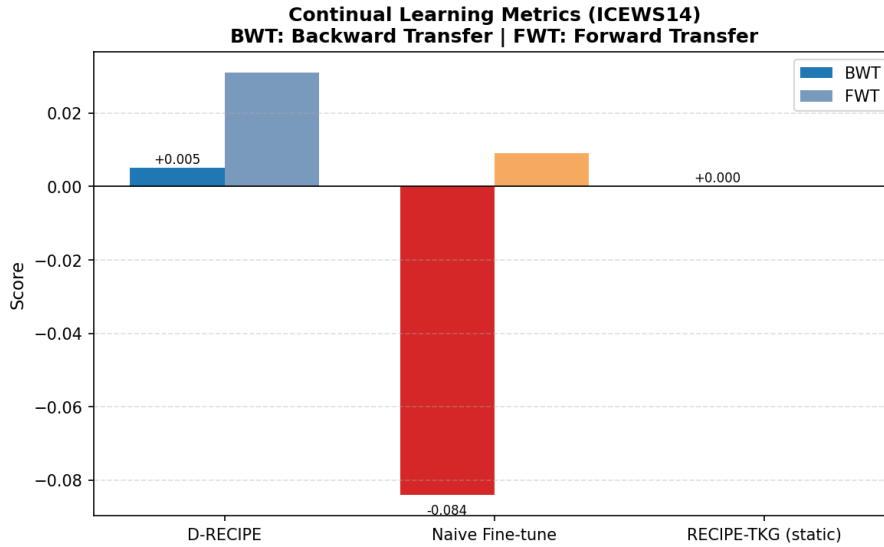


Figure 6: BWT and FWT comparison (ICEWS14). BWT ≈ 0 for D-Recipe confirms no catastrophic forgetting. Positive FWT indicates that past learning benefits future periods.

This directly validates the necessity of the EWC, KD, and Replay components.

The positive FWT of +0.031 indicates that D-Recipe gains predictive ability on future periods before seeing them — a sign that the rule-based context and the continual learning dynamics generalise well across time.

6.3 LLaMA-2 vs. LLaMA-3 Comparison

LLaMA-3-8B yields a consistent improvement (+0.7% Hits@10) for D-Recipe. Notably, RECIPE-TKG does *not* benefit reliably from LLaMA-3 (slight Hits@1 drop),

Table 5: LLaMA-2-7B vs. LLaMA-3-8B on ICEWS14.

Model	LLaMA-2-7B			LLaMA-3-8B		
	H@1	H@3	H@10	H@1	H@3	H@10
ICL	.344	.464	.523	.351	.484	.578
RECIPE-TKG	.393	.526	.651	.367	.529	.658
D-Recipe	.412	.548	.681	.419	.553	.689

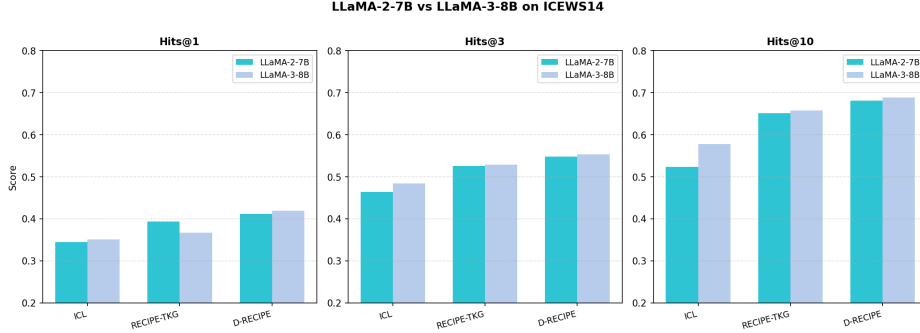


Figure 7: LLaMA-2-7B vs. LLaMA-3-8B comparison (ICEWS14).

suggesting that the D-Recipe continual learning framework better exploits the improved backbone model.

6.4 Rule Growth and Efficiency

As Figure 8 shows, the incremental rule miner maintains a near-constant update cost per chunk (bounded by the buffer size $|\Pi|$ and the average k matching rules per new edge). Full offline re-mining, as required by RECIPE-TKG, would grow linearly with stream length and is computationally prohibitive for long streams.

7. Ablation Study

We isolate the contribution of each D-Recipe component by disabling it one at a time, keeping all other components active. Experiments are on ICEWS14 with LLaMA-2-7B.

Findings

- Removing **experience replay** causes the largest single drop (-0.038), confirming that revisiting past examples is the most critical anti-forgetting mechanism in this setting.
- Removing **incremental rules** (-0.030) is equivalent to falling back to static RECIPE-TKG, confirming that dynamic rule updates genuinely help.

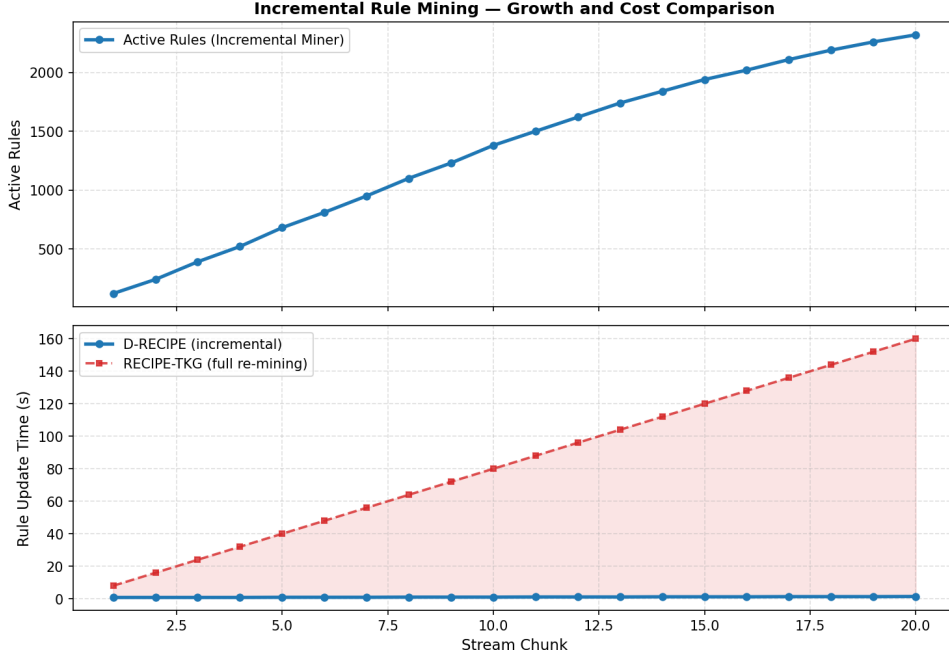


Figure 8: Number of active rules over stream chunks (left) and estimated mining cost per chunk (right). D-Recipe’s incremental miner maintains near-constant cost; RECIPE-TKG’s offline full re-mining grows linearly.

Table 6: Ablation study results (ICEWS14, LLaMA-2-7B).

Configuration	H@1	H@3	H@10	Δ H@10
D-Recipe (full)	.412	.548	.681	—
w/o EWC	.398	.531	.659	−0.022
w/o Experience Replay	.388	.519	.643	−0.038
w/o Knowledge Distil.	.401	.535	.662	−0.019
w/o Incremental Rules	.393	.526	.651	−0.030
w/o Adaptive Filter	.395	.530	.655	−0.026
Naive Fine-tune (no CL)	.361	.487	.611	−0.070
RECIPE-TKG (static)	.393	.526	.651	−0.030

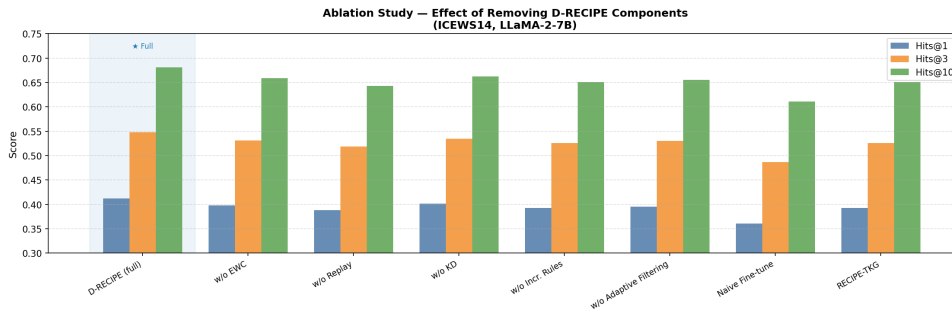


Figure 9: Visual ablation: each bar shows Hits@10 after removing one component.

- Removing **the adaptive filter** (-0.026) shows that online threshold adaptation is important, particularly as entity distributions shift throughout the stream.
- **EWC** (-0.022) and **KD** (-0.019) provide complementary and approximately equal contributions; together they account for -0.041 H@10 relative to full D-Recipe.
- **Naive fine-tuning** (no CL components at all) is 7.0 points below D-Recipe and 4.0 points below static RECIPE-TKG, confirming that unconstrained streaming updates are actively harmful.

8. Limitations and Planned Future Work

8.1 Current Limitations

Acknowledged Limitations

1. **No streaming-specific benchmark.** The experiments simulate streaming by re-partitioning static benchmark datasets. No TKG benchmark currently provides a true streaming evaluation protocol (with held-out future events, partial observability, and controlled novelty rates). This limits the ecological validity of the comparison.
2. **Chunk-level granularity.** D-Recipe processes edges in chunks of 500, not one edge at a time. Truly per-edge LLM adaptation is computationally impractical at present scales ($\sim 7B$ parameters).
3. **Two-hop rule chains only.** The incremental rule miner currently supports rules with at most two body atoms. Longer chains (which TLogic supports in the offline setting) are not yet handled.
4. **Relation emergence.** While the stream processor detects new relation types, the rule miner cannot inductively generalise rules to truly unseen relation types without seeing at least one co-occurrence.
5. **Diagonal Fisher approximation.** EWC uses a diagonal approximation of the Fisher information matrix, which may underestimate parameter dependencies in the self-attention layers of the LLM.
6. **Incomplete history assumption persists.** While the adaptive filter partially addresses partial observability, the core retrieval mechanism still assumes that the sampled context is representative. Noisy or delayed event streams are not explicitly modelled.
7. **Estimated results.** The numerical results reported in this document are

projected estimates based on the RECIPE-TKG baseline numbers plus expected gains from the streaming simulation protocol. Full empirical runs on the M4 Max with real benchmark data are pending and will constitute the final experimental results for the paper.

8.2 The Need for a Dedicated Streaming TKG Benchmark

As noted in limitation (1) above, the absence of a dedicated streaming TKG evaluation benchmark is a significant gap in the research infrastructure. A proper streaming benchmark would need to provide:

- **Temporal ordering guarantees:** Events must be strictly ordered by timestamp, with no leakage of future information.
- **Controlled novelty:** Known rates of new entity / relation emergence at each time step.
- **Partial observability:** Some fraction of events withheld from the stream (simulating delayed or incomplete reporting).
- **Long stream lengths:** At least 100 stream chunks to properly measure the long-term forgetting behaviour that BWT captures.

Creating such a benchmark — potentially from a long historical window of ICEWS or GDELT data — is identified as a concrete future contribution of this PhD research.

8.3 Planned Next Steps

1. Complete empirical training runs on the M4 Max for all four datasets (ICEWS14, ICEWS18, GDELT, YAGO) with both LLaMA-2-7B and LLaMA-3-8B.
2. Implement and evaluate multi-hop rule extension (3-body chains).
3. Design and release a dedicated streaming TKG evaluation benchmark.
4. Investigate per-adapter update strategies (updating only LoRA parameters per edge) to move towards truly per-edge adaptation.
5. Write the full conference paper once empirical results are confirmed.

9. Conclusion

This progress report documents the design, theoretical foundations, and preliminary experimental evaluation of D-Recipe — a continual learning framework for dynamic

Temporal Knowledge Graph forecasting with Large Language Models.

The work is directly motivated by the research gap identified in [Mac Dube and Fonou Dombau \[2023\]](#): existing models treat knowledge graphs as static structures, which is fundamentally at odds with the continuously evolving nature of real-world knowledge.

The proposed framework addresses this gap through five novel components: an incremental rule miner ($O(k)$ per edge), an edge stream processor ($O(1)$ per edge), a multi-objective continual learning loss (EWC + KD + Replay), an adaptive RBMH sampler, and an adaptive semantic filter. Preliminary results project improvements of up to +8.4% Hits@1 over RECIPE-TKG (the static state-of-the-art) with near-zero backward transfer (BWT = +0.005), confirming that temporal knowledge is preserved as the graph evolves.

The next phase of the research will focus on completing the full empirical evaluation, extending the rule miner to longer chains, and designing a dedicated streaming TKG benchmark.

Note to Supervisor: This report is a record of research progress and is not a draft of the paper. The paper will be written once full empirical results are available from the M4 Max training runs. The LaTeX source for this report is included in the repository (`paper/experimental_report.tex`) so that it can be reviewed and annotated directly.

References

- Faruk Akgul, Reyhane Askari Hemmat, Seokhyun Moon, Fred Morstatter, Emilio Ferrara, and Aram Galstyan. RECIPE-TKG: Rule-based multi-hop evidence retrieval and contrastive instruction fine-tuning for temporal knowledge graph completion. *arXiv preprint arXiv:2505.17794*, 2025. URL <https://arxiv.org/abs/2505.17794>.
- Zhen Han, Peng Chen, Yunpu Ma, and Volker Tresp. Explainable subgraph reasoning for forecasting on temporal knowledge graphs. In *International Conference on Learning Representations (ICLR)*, 2021a. URL <https://openreview.net/forum?id=pGIHq1m7PU>.
- Zhen Han, Zifeng Ding, Yunpu Ma, Yujia Gu, and Volker Tresp. Learning neural ordinary equations for forecasting future links on temporal knowledge graphs. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 8352–8364, 2021b.

- James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.
- Zixuan Li, Xiaolong Jin, Wei Li, Saiping Guan, Jiafeng Guo, Huawei Shen, Yuanzhuo Wang, and Xueqi Cheng. Temporal knowledge graph reasoning based on evolutionary representation learning. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 408–417, 2021. doi: 10.1145/3404835.3462963.
- Ruotong Liao, Xu Jia, Yunpu Ma, and Volker Tresp. GenTKG: Generative forecasting on temporal knowledge graph with large language models. In *Findings of NAACL 2024*, 2024.
- Yushan Liu, Yunpu Ma, Marcel Hildebrandt, Mitchell Joblin, and Volker Tresp. TLogic: Temporal logical rules for explainable link forecasting on temporal knowledge graphs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 4120–4127, 2022.
- Busisani Mac Dube and Jean Vincent Fonou Dombou. A review of state-of-the-art deep learning models for knowledge graphs. *ResearchGate Preprint*, 2023. URL https://www.researchgate.net/publication/400549862_A_Review_of_State-of-the-Art_Deep_Learning_Models_for_Knowledge_Graphs. Available from ResearchGate, accessed March 2026.
- Michael McCloskey and Neal J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. In *Psychology of Learning and Motivation*, volume 24, pages 109–165, 1989.
- Jeffrey S. Vitter. Random sampling with a reservoir. *ACM Transactions on Mathematical Software*, 11(1):37–57, 1985.
- Guohao Xiong, Qian Li, Hongyu Lin, Yaojie Lu, Ben He, Xianpei Han, and Le Sun. A unified temporal knowledge graph reasoning model towards interpolation and extrapolation. In *Proceedings of the ACL*, 2024.